

## I. The Variable Lifecycle & Core Problem

### 1. Standard vs. Static Variables

Establish the baseline of how memory persists:

- **Standard (Automatic) Variables:** When a function is called, standard variables are created. When the function finishes and returns, those variables are destroyed, and their data is permanently lost.
- **Static Variables:** A static variable survives the death of the function. It is initialized only once, and it retains its value in memory between multiple function calls.

### 2. The Core Problem (The Blind System Call)

Explain why persistence is needed by talking about the operating system:

- **The read() Function:** The OS doesn't know what a "line" is. It only knows how to grab a blind chunk of bytes based on the `BUFFER_SIZE`.
- **The Unpredictable Chunk:** Because `BUFFER_SIZE` can be anything (1 byte, 42 bytes, or 10,000 bytes), a single `read()` might give us half a line, exactly one line, or three and a half lines all glued together.

### 3. The get\_next\_line Engine (Accumulate, Extract, Preserve)

Tie the static variable and the chunking problem together:

- **The Delimiter:** We are hunting for the newline character (`\n`) or the End of File (EOF).
- **Accumulation:** We keep calling `read()` and stacking those chunks together until our delimiter finally appears in the sequence.
- **Extraction:** Once we find the `\n`, we slice off everything from the beginning up to that delimiter and hand it back to the user as the final line.
- **The Leftover (The "Aha!" Moment):** This is the most crucial part of your defense. Because the last chunk might have contained the end of Line 1 and the beginning of Line 2, we cannot just throw the rest of the string away. We use the **static variable** to preserve those leftover characters so that the next time `get_next_line` is called, it already has a head start on Line 2.

## II. Architecture: Public API vs. Internal Plumbing

### 1. The Public API (What goes in the .h file)

- The header file acts as your project's public menu. When another program (like a `main.c` file for testing) includes `get_next_line.h`, it only needs to know how to use the primary tool: `get_next_line()`.
- The utility functions (`ft_strlen`, `ft_strchr`, `ft_strjoin`) live in a completely separate file (`get_next_line_utils.c`). Because your main `get_next_line.c` file has to "reach across" to another file to call them, the compiler requires their prototypes ("name tags") to be listed in the shared header so the files can talk to each other.

### 2. The Internal Plumbing (What stays out of the .h file)

- Your helper functions (`clean_reserve`, `extract_line`, `read_to_reserve`) are the internal, private gears of your engine. They do not need to be prototyped in the header because they are defined inside the exact same file (`get_next_line.c`) where they are called.
- Because you placed them physically *above* the main `get_next_line()` function, the compiler reads them from top to bottom. By the time it reaches your main function, it already knows exactly what they are and how they work.
- No outside program has any business calling `clean_reserve` directly, so it shouldn't be advertised in the public header.
- *Note on the Norm:* This concept is closely tied to why the strict 42 Norm usually requires internal helper functions to be declared as **static**—it acts as a deadbolt, locking those functions exclusively inside that specific `.c` file so they can never conflict with functions in other parts of a massive program.

## III. Memory Management & Leak Defense

Be prepared to explain every `malloc` call in the program. You have built a layered defense against memory leaks that relies on the functions working together as a team.

### 1. clean\_reserve: The Direct Check

- You call `malloc` to create the new leftover string.
- Immediately after, you check `if (!str)`.
- If it fails, you `free(reserve)` and return `NULL`, ensuring the old static memory is destroyed before the function collapses.

## 2. extract\_line & get\_next\_line: The Safety Net

- `extract_line` returns `NULL` if `malloc` fails, but it doesn't free `reserve`. This is a deliberate design choice.
- If `extract_line` fails, it signals the main function by passing that `NULL` back up.
- **The Defense Explanation:** "I delegated the cleanup. The main `get_next_line` function has a safety net waiting. It catches the failure, frees the static `reserve` to prevent leaks, resets it, and gracefully exits:"

```
line = extract_line(reserve);
if (!line)
{
    free(reserve);
    reserve = NULL;
    return (NULL);
}
```

## 3. read\_to\_reserve: The Silent Bypass

- There is no explicit `if (!bucket)` check right after the `malloc`.
- **The Defense Explanation:** "In `read_to_reserve`, I placed `bucket` as the primary condition in the `while` loop (`while (bucket && !ft_strchr(reserve, '\n') && b > 0)`). If the allocation fails, `bucket` is `NULL`, which short-circuits the entire loop. The code jumps down to safely call `free(bucket)`; (in C, calling `free(NULL)` is valid and does nothing). Because `b` remains initialized to 1, the `if (b == -1)` check is skipped, and the function safely returns the existing `reserve` string without altering it or leaking any new memory. The system simply falls back to trying to extract a line from whatever data was already stored."

# IV. Live Coding Modification (Custom Separator)

### The Quick-Swap Strategy

If the evaluator asks you to modify the project to support a custom separator character (e.g., a comma `,`), the fastest and most effective way to satisfy the requirement under pressure is to manually replace the hardcoded `\n` characters.

You must change exactly **5 locations** inside `get_next_line.c`:

1. **1 spot** in `read_to_reserve`
2. **3 spots** in `extract_line`
3. **1 spot** in `clean_reserve`

Jump into the file, delete `'\n'`, and type `','` (or whatever character they request) in those 5 specific spots.

### How to Prove It Works (The Concrete Example)

To complete the 10-minute task and get full points, you must demonstrate the modification with a concrete test.

1. **Create a test file:** Make a file called `test.txt` with comma-separated words:

```
apple,banana,cherry,
```

2. **Write a basic main.c:**

```
#include "get_next_line.h"
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    int fd = open("test.txt", O_RDONLY);
    char *line;

    while ((line = get_next_line(fd)) != NULL)
    {
        printf("Extracted: %s\n", line);
        free(line);
    }
    close(fd);
    return (0);
}
```

3. **Compile and Run:** When the terminal successfully prints `apple,,`, then `banana,,`, then `cherry,,` one by one, you have passed the live test.